Ders 3 Genişletilmiş Notu: Asimptotik Gösterimler: Büyük-O, Büyük-Omega, Büyük-Teta ve Büyüme Dereceleri

Ders 3 Genişletilmiş Notu: Asimptotik Gösterimler: Büyük-O, Büyük-Omega, Büyük-Teta ve Büyüme Dereceleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Geçen Haftanın Tekrarı: Temel İşlemler ve Sabitler
- 2. Büyüme Derecesini Etkilemeyenler
- 3. Büyük-O Gösterimi ve Slayttaki Örnekler
- 4. Büyük-O Kuralları: En Yakın Üst Sınır
- 5. Asimptotik Analiz ve En Kötü Durum
- 6. Yaygın Büyüme Dereceleri ve Örnek Algoritmalar
- 7. Önek Ortalamaları: $O(n^2)$'den $O(n)$'e
- 8. Asimptotik Gösterim Kuralları
- 9. Büyük-Omega ve Büyük-Teta
- 10. Logaritma ve Üs Kuralları
- 11. Sabit Çarpanlar Pratikte Önemlidir
- 12. Tahtada Çözülen Örnekler
- 13. Insertion Sort'un Sınırları

1. Geçen Haftanın Tekrarı: Temel İşlemler ve Sabitler

Hoca derse, bu haftanın konusunun asimptotik gösterimler (asymptotic notations) ve büyüme dereceleri (order of growth) olduğunu söyleyerek başladı. Büyük-O (Big-O), Büyük-Omega (Big-Omega) ve Büyük-Teta (Big-Theta) gösterimleri öğrenilecek, böylece bir algoritmanın teorik olarak nasıl analiz edildiği daha derinlemesine görülecekti.

Geçen hafta bir algoritmadaki temel işlemler (primitive operations) en kötü durum (worst case) için sayılmıştı. Girdi sayısı n kabul edilmişti. Diziden en büyük elemanı bulan `arrayMax` algoritması için en kötü durumda $7n-1$ temel işlem bulunmuştu. Bu sayı bir fonksiyonla ifade edilir. Slaytta $T(n)$ kullanılıyor; T zamanın (time) baş harfidir. Hoca $f(n)$, $g(n)$ gibi başka adların da kullanılabileceğini, bazı kitaplarda karşılaştırma (comparison) sayısı anlamında $C(n)$ yazıldığını söyledi.

Bu işlemler içinde en hızlı çalışanın süresi a, en yavaş çalışanınki b sabiti olsun. O zaman maksimum bulma algoritmasının çalışma süresi şu aralıktadır:

$$a(7n-1) \leq T(n) \leq b(7n-1)$$

Hocaya göre burada a ve b'nin bizim için bir önemi yoktur. Amaç zaten bu sabitlerden kurtulup, girdi sayısına bağlı bir ifadeyi teorik olarak elde etmektir. $7n-1$ doğrusal (linear) bir fonksiyondur. İleride logaritmik, karesel (quadratic) ve kübik (cubic) büyüme dereceleri de görülecektir.

Donanım (hardware) ve yazılım (software) ortamının değişmesi bu sabitleri doğrudan etkiler. Ama büyüme derecesi fonksiyonunu etkilemez. Hoca bunu bir avantaj olarak sundu: Bir algoritmayı büyüme derecesiyle ifade ettiğimizde, sabitlerden bağımsız olarak onun zaman karmaşıklığını (time complexity) ya da zaman verimliliğini (time efficiency) ifade etmiş oluruz. Bu yüzden ders boyunca büyüme derecesi kavramı üzerinde yoğunlaşılacaktır.

2. Büyüme Derecesini Etkilemeyenler

Büyüme derecesi iki şeyden etkilenmez:

- Sabit çarpanlar (constant factors).
- Daha düşük dereceli terimler (lower-order terms).

Slayttaki ilk örnek $10^2n + 10^5$ idi. Burada 10^5'in de, 10^2 çarpanının da önemi kalmaz; fonksiyon doğrusaldır. İkinci örnek $10^5n^2 + 10^8n$ idi. Burada sabitleri atmanın ötesinde düşük dereceli terimi de atmak gerekir. En büyük derece n^2 olduğu için n'li terimden ve sabitlerden kurtuluruz. Bu fonksiyon Büyük-O cinsinden $O(n^2)$ olarak ifade edilir.

Hoca Büyük-O'nun üst sınıra (upper bound) karşılık geldiğini, bunun ilerleyen slaytlarda ve tahtada gösterileceğini söyledi. Büyük-O gösteriminin asıl amacı, zaman ve yer (space) bakımından sınırları bulmaktır. Gösterim, sabit çarpanların ve düşük dereceli terimlerin temizlenmesini sağlar.

3. Büyük-O Gösterimi ve Slayttaki Örnekler

3.1 Tanım

$f(n)$ ve $g(n)$ iki fonksiyon olsun. Öyle bir pozitif c sabiti ve bir n_0 değeri varsa ki $n \geq n_0$ olan her n için

$$f(n) \leq c\, g(n)$$

sağlanıyorsa, $f(n)$'yi $g(n)$ cinsinden ifade edebiliriz: $f(n) \in O(g(n))$. Hocanın deyişiyle, eşitsizlik "n'in belli bir değerinden itibaren, sabit bir sayıdan itibaren" sağlanmalıdır. $c > 0$ olmalıdır, n_0 da pozitif bir sayı seçilir.

3.2 Örnek: $2n + 10 \in O(n)$

Slayttaki üç fonksiyonlu grafikte ortadaki $2n+10$ bir algoritmanın karmaşıklığı olarak düşünülür. Soru şudur: $2n + 10$, n cinsinden ifade edilebilir mi? Burada $f(n) = 2n + 10$, $g(n) = n$'dir.

- Eşitsizlik kurulur: $2n + 10 \leq cn$.
- $2n$ karşıya atılır: $10 \leq (c - 2)n$, yani $n \geq \dfrac{10}{c-2}$ (ara adım eklendi).
- $c = 3$ seçilirse $n \geq 10$ olur. Yani $c = 3$, $n_0 = 10$ için eşitsizlik sağlanır.

Bu sabitlerle belli bir noktadan sonra bütün n'ler için eşitsizlik sağlandığından $2n + 10 \in O(n)$'dir. Hoca bu örneğin deftere, gerekirse çizimiyle birlikte not edilmesini istedi.

Hoca, asimptotik gösterimde üç ifade biçimi olduğunu hatırlattı: Büyük-O üst sınır, Büyük-Omega alt sınır (lower bound), Büyük-Teta sıkı sınır (tight bound). Ağırlıklı olarak Büyük-O kullanılır.

3.3 Örnek: $n^2 \notin O(n)$

Slayttaki ikinci grafikte n, $10n$, $100n$ ve n^2 fonksiyonları çizilmişti. Soru: n^2'nin üst sınırı n olabilir mi?

- $n^2 \leq cn$ yazılır.
- İki taraf n'e bölünür: $n \leq c$.

c bir sabit olmak zorundadır, oysa n bir değişkendir. Hiçbir sabit c, belli bir n'den itibaren bütün n'ler için $n \leq c$'yi sağlamaz. Hoca, n'in yerinde 5 gibi bir sayı olsaydı bunun rahatça söylenebileceğini ekledi. Bu yüzden n^2, $O(n)$ değildir.

> **Hocanın vurgusu:** n^2'nin üst sınırının n olamayacağı polinomlara bakınca zaten görülüyor. Ama ispat açısından bu analizi yapmanız sizden bekleniyor.

3.4 Diğer slayt örnekleri

Hoca kalan örneklerin üzerinden hızlıca geçti:

- $7n - 2 \in O(n)$**:** $7n - 2 \leq cn$. $c = 7$, $n_0 = 1$ ile sağlanır.
- $3n^3 + 20n^2 + 5 \in O(n^3)$**:** Çözümleme aynı şekilde yapılır. Hoca sabitlerin değiştirilebileceğini söyledi: c "7 ya da 4 olmak zorunda değil"; farklı c seçildiğinde n_0 da değişir, ama o noktadan sonra eşitsizlik sağlanıyorsa sorun yoktur. Python ile kontrol: $c = 4$ için en küçük $n_0 = 21$, $c = 7$ için en küçük $n_0 = 6$'dır.
- $3 \log n + 5 \in O(\log n)$**:** $f(n) = 3 \log n + 5$, $g(n) = \log n$. $c = 8$, $n_0 = 2$ seçildiğinde o noktadan itibaren eşitsizlik sağlanır ($3 \log n + 5 \leq 8 \log n \iff \log n \geq 1$; ara adım eklendi).

Hoca Büyük-O'nun ne yaptığını şöyle özetledi: Önce sabitlerden, sonra daha düşük dereceli terimlerden kurtuluruz. $3n^3 + 20n^2 + 5$'te n^2 terimi vardır. n sonsuza giderken n^2'nin n^3 yanında hiçbir önemi kalmaz. Bu yüzden düşük dereceli terim olarak atılır. Sabitler de atılınca geriye n^3 kalır. Hoca bunun limit alma yöntemiyle de yapılabileceğini ve ileride görüleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Limit yönteminde $L = \lim_{n \to \infty} f(n)/g(n)$ hesaplanır. $L = 0$ ise f, g'den yavaş büyür ve $f \in O(g)$ olur. $0 < L < \infty$ ise ikisi aynı derecededir ve $f \in \Theta(g)$ olur. $L = \infty$ ise $f \in \Omega(g)$ olur ama $f \notin O(g)$.

3.5 Büyüme oranı tablosu

$f(n) \in O(g(n))$ ifadesi, "$f(n)$'nin büyüme derecesi $g(n)$'ninkinden fazla olamaz" demektir. Slayttaki tablo bunu üç durum için gösteriyordu:

Durum	$f(n)$ is $O(g(n))$	$g(n)$ is $O(f(n))$
$g(n)$ daha hızlı büyüyor	Evet	Hayır
$f(n)$ daha hızlı büyüyor	Hayır	Evet
Aynı büyüme derecesi	Evet	Evet

İkisi aynı büyüme derecesine sahipse iki gösterim de kullanılabilir.

4. Büyük-O Kuralları: En Yakın Üst Sınır

$f(n)$, d. dereceden bir polinomsa $f(n) \in O(n^d)$'dir. Yani düşük dereceli terimler düşürülür, sabitlerden de kurtulunur.

Slaytta "$2n$ için $O(n)$ demek, $O(n^2)$ demekten daha uygundur" yazıyordu. Hoca ilk başta örneğin neden böyle yazıldığını sorguladı, sonra çok da yanlış olmadığını, şunu göstermek için verildiğini söyledi: $2n$'nin üst sınırı n^2 de, n^3 de, n^4 de olabilir. Bunların hepsi birer üst sınırdır. $2n \in O(n^2)$ demek yanlış değildir. Ama algoritmanın karmaşıklığını doğru ifade etmek için fonksiyona **en yakın** üst sınırı seçmek gerekir. Bu yüzden n^2 yerine n seçilir.

İkinci örnek: $3n + 5 \in O(n)$ demek, $3n + 5 \in O(3n)$ demekten daha uygundur. Amaç sabitlerden kurtulmak olduğu için, büyüme derecesinin başına bir sabit yazmaktan kaçınılır. $O(3n)$ yazılmaz.

> **Hocanın vurgusu:** Bu soruları yıllardır soruyoruz ve bunlarla ilgili mutlaka sınavda soru geliyor. Ama genelde tam oturmadığı için önemli derecede hata yapılabiliyor. Sorunuz varsa sormaktan çekinmeyin.

5. Asimptotik Analiz ve En Kötü Durum

Hoca slayttaki asimptotik analiz maddelerini öğrencilere Türkçeye çevirtti.

Soru: Slayttaki "Asimptotik analizi uygularken..." ifadesi Türkçe ne diyor? **Cevap:** Sabitler ve düşük dereceli terimler zaten düşeceğinden $7n - 1$'i $O(n)$ olarak ifade edebiliriz.

Soru: "We find the worst case number of primitive operations" ne demek istiyor? **Cevap:** En kötü durumda algoritmanın, örneğin `for` döngüsünün, ne kadar çalışabileceğini hesaplamamızı istiyor.

Hoca burada kritik bir noktanın altını çizdi. $7n - 1$ işlem, en kötü durum senaryosunda çalışacak işlem sayısıdır. Girdinin dizilimine, sırasına bağlı olarak algoritma farklı sayıda işlem yapabilir. Bunun için en iyi durum (best case) ve ortalama durum (average case) kavramları kullanılır. Her algoritmanın farklı durumları vardır ve asimptotik analiz bu durumlara göre yapılır.

> **Hocanın vurgusu:** Aksi belirtilmedikçe sizden en kötü durum analizi bekleniyor. En iyi ya da ortalama durum sorulursa hesabı ona göre yapmanız gerekiyor.

6. Yaygın Büyüme Dereceleri ve Örnek Algoritmalar

Hoca, algoritma analizinde sık karşılaşılan büyüme derecelerini tek tek öğrencilere örnek sorarak geçti.

- $O(1)$**, sabit: Soru:** Karmaşıklığı $O(1)$ olan bir algoritma? **Cevap:** Sıralı bir dizide en büyük elemanı bulmak. Hoca bunu, en küçük elemanı bulmakla birlikte basit ve güzel bir örnek olarak kabul etti.
- $O(\log n)$**, logaritmik:** İkili arama (binary search). İkiye bölerek ilerlenir. Örneğin 1000 sayı içinde arama yapıldığında, en kötü durumda genelde onuncu adımda aranan sayının var olup olmadığı söylenebilir. $\log n$, doğrusal n'e göre daha az karmaşıktır.
- $O(n)$**, doğrusal: Soru:** Doğrusal bir algoritma örneği? **Cevap:** Bir dizide verilen bir tamsayıdan büyük kaç sayı olduğunu bulan algoritma. Hoca dizinin sırasız verildiğini kabul etti. Sıralı olsaydı da arama sıralı (sequential) yapılırsa en kötü durumda yine $O(n)$ olurdu. Hoca bazı yaklaşımların daha hızlı çözüm üretebileceğini, ama şu an yalnız örnek üzerinden gidildiğini ekledi.
- $O(n \log n)$**:** Heap sort. Hoca sıralama algoritmalarının bu derecede sık karşımıza çıktığını, ama başka problemlerde de görüleceğini söyledi. Bu problemlere bulunan ilk çözümler genelde n^2 boyutundaydı. Örneğin sıralamada selection sort n^2 karmaşıklığa sahiptir. Zamanla quick sort ve merge sort gibi algoritmalar çıkınca, durumlara göre değişmekle birlikte, büyüme $n \log n$'e indirilebildi. $n \log n$, n ile n^2 arasında kalır.
- $O(n^3)$**, kübik:** Matris çarpımı. Hoca bunu her zaman kullanılan basit bir örnek olarak onayladı.
- $O(2^n)$**, üstel:** n^3'ten daha büyük bir büyüme derecesidir. **Soru:** Matematikte 2^n'e karşılık gelen sık bir örnek? **Cevap:** Alt küme sayısını bulmak. n elemanlı bir kümenin, boş küme ve bütün elemanların dahil olduğu küme ile ikili, üçlü kombinasyonlar hesaba katılınca, 2^n alt kümesi vardır.
- $O(n!)$**, faktöriyel: Soru:** $n!$'in büyüme derecesi 2^n'den büyük mü, aynı mı, küçük mü? **Cevap:** Daha büyüktür. $5! = 120$, $2^5 = 32$. n büyüdükçe aradaki fark açılır. **Soru:** $n!$ matematikte neye karşılık gelir? **Cevap:** Bir kümenin permütasyonlarına. Hocanın örneği: n kişilik bir grubu kaç farklı şekilde sıraya dizebileceğinizi sorarsanız cevap $n!$'dir.

Hoca bunların bilgisayar bilimlerinde en sık kullanılan büyüme dereceleri olduğunu, problemler farklılaştıkça başka derecelerle de karşılaşılabileceğini söyledi. Sıralama şöyledir:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$$

6.1 Büyüme derecesinin maliyete etkisi

Slayttaki tabloda n değerleri için her fonksiyonun aldığı değerler veriliyordu. $\log n$ çok yavaş büyürken n, verilen değer kadar büyür. $n \log n$, n'e göre biraz daha hızlı artar, ama n^2'ye göre oldukça makul bir değişim gösterir.

Hocanın örneği: 256 elemanlı bir diziyi sıralarken

- selection sort ile $n^2 = 256^2 = 65536$ işlem,
- heap sort, merge sort gibi algoritmalarla $n \log_2 n = 256 \cdot 8 = 2048$ işlem

yapılır. Oran $65536/2048 = 32$'dir. Hoca "yaklaşık 30 kat" dedi; tam değer 32 kattır.

6.2 Üstel algoritmalar ve kaba kuvvet

Hoca n^3 ve 2^n'e gelindiğinde, bazı problemlerin hâlâ polinom zamanda çözülme şansı olmadığını söyledi (bu cümle için bkz. Ders sonrası notlar).

Soru: 2^n, $n!$ gibi büyüme derecelerine sahip algoritmalar, geçen hafta konuşulan algoritmik yaklaşımlardan hangisine karşılık gelir? Örneğin şifre çözmede ilk etapta hangi yaklaşım kullanılır? **Cevap:** Kaba kuvvet (brute force).

Kaba kuvvet bütün seçeneklerin tek tek denendiği yaklaşımdır. Hocanın örneği: Şifre 3 haneliyse yaklaşık 1000 deneme gerekir. 4 haneliyse bu büyür, 5 haneliyse ve karakter kombinasyonları da varsa daha da fazla olur. Normalde kaba kuvvet tercih edilmez, ama bazı problemler için ondan iyi bir çözüm olmayabilir. Şifre kırmaya çalışanlar da bu yüzden kaba kuvvete başvurur. Hoca, bu nedenle eskiden çok daha kısa anahtarlar kullanılırken bugün 256 bit, hatta 512 bit şifrelemeden bahsedildiğini söyledi. Bu, karmaşıklığı artırarak güvenliği yükseltir. Bedeli de vardır: Kodlama (encoding) ve kod çözme (decoding) için güçlü donanım gerekir. Şifreleme türüne göre bazen kod çözme, bazen kodlama daha güçlü bir makine ister.

Hoca, büyüme derecelerinin algoritma verimliliğinde çok önemli bir ölçüt olduğunu ve iyi bilinmesi gerektiğini söyledi. Slayttaki çalışma süreleri tablosuna ise kritik olmadığı için değinmedi.

7. Önek Ortalamaları: $O(n^2)$'den $O(n)$'e

Slayttaki örnek, bir dizide i. elemana kadar olan elemanların ortalamasını bulmaktır (prefix averages). Hoca, çözüm şeklinin algoritmik karmaşıklığı çok etkilediğini bu örnekle gösterdi. İki yaklaşım vardır.

Birinci yaklaşım: İç içe iki `for` döngüsü kullanılır. Dış döngünün her adımında, o noktaya kadar olan sayılar iç döngüde baştan bir kez daha toplanır. Hoca bunu "gayet ilkel bir yöntem" olarak niteledi. İşlemler toplandığında maliyet $O(n^2)$ çıkar.

> **Ek bilgi (derste anlatılmadı):** İç döngü i. adımda i eleman topladığı için toplam iş $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ civarındadır. Bu ifadenin en yüksek dereceli terimi $n^2/2$ olduğundan maliyet $O(n^2)$'dir.

Hoca bu yaklaşımı tümden yanlış saymadı. Algoritmaya yeni başlayan birinin böyle bir çözüm üretmesi normaldir; önemli olan ilk etapta doğru sonucu veren bir yöntem bulmaktır. Ama karmaşıklık ortaya konduktan sonra "bundan daha iyisi olabilir mi?" sorusunun peşine düşmek gerekir. Bugüne kadar ortaya atılan algoritmalardaki iyileştirmeler, karmaşıklık incelemesi sonucunda daha iyi bir çözüm olabileceğinin görülmesiyle gerçekleşmiştir.

İkinci yaklaşım: İç içe döngü yerine, her adımda o ana kadarki toplam tutulur. Yeni eleman bu toplama eklenir ve toplam, o anki sıra numarasına bölünür. Böylece bir önceki adımın sonucu ekstradan bir yerde saklanmış olur ve o noktaya kadar yapılan işler tekrar tekrar yapılmaz. Tek döngü yeter ve maliyet $O(n)$'e iner. Hoca bunun çok zor bir bakış açısı olmadığını, ama akla gelmeyebileceğini söyledi.

Hoca bu fikrin dersin ilerleyen haftalarında görülecek dinamik programlama (dynamic programming) yaklaşımına çok benzediğini söyledi: O ana kadarki sonuçları saklayarak aynı işi tekrar tekrar yapmazsınız.

Soru: Gerideki hesaplamaları kullanarak ileri doğru ilerlediğimiz, birbiriyle ilişkili sayılardan oluşan meşhur sayı dizisi hangisidir? **Cevap:** Fibonacci dizisi.

Fibonacci dizisinde her seferinde önceki iki sayı toplanıp sonuç yazılır. Bir sonraki adımda daha gerideki sayı bırakılır, zaten hesaplanmış olan son iki sayı alınıp devam edilir. Her seferinde

baştan hesaplasaydık maliyet çok yüksek olurdu. Bunun yerine elde edilen sonuç bir yerde saklanır. Önek ortalamalarında da aynı fikirle $O(n^2)$'den $O(n)$'e inilmiş olur.

8. Asimptotik Gösterim Kuralları

Aradan sonra hoca kurallara geçti. $f(n)$, d. dereceden bir polinom olsun: $f(n) = a_0 + a_1 n + a_2 n^2 + \cdots + a_d n^d$. Bu durumda $f(n) \in O(n^d)$'dir.

- **Toplam ve çarpım kuralı:** $d(n) \in O(f(n))$ ve $e(n) \in O(g(n))$ ise
  - $d(n) + e(n) \in O(f(n) + g(n))$,
  - $d(n)\, e(n) \in O(f(n)\, g(n))$.
- **Geçişlilik (transitivity):** $d(n) \in O(f(n))$ ve $f(n) \in O(g(n))$ ise $d(n) \in O(g(n))$.
- **Logaritma kuralı:** $p(n)$ bir polinomsa $\log p(n) \in O(\log n)$.

Hoca toplam ve çarpım kuralının çok sık kullanılmadığını, ama iki farklı fonksiyonun birlikte nasıl ifade edileceğini bilmek için not edilmesini istedi.

Logaritma kuralı vesilesiyle hoca, birçok logaritmik ifadede tabanın yazılmadığını söyledi. Genellikle 2 tabanı kastedilir; bazı problemlerde tabanı doğrudan 2 olarak yazacaklardır. Ama önemli olan tabanın 2 ya da 3 olması değil, fonksiyonun logaritmik büyüdüğünü göstermektir. Logaritmik büyüyen bir fonksiyon, n'e göre çok daha yavaş büyür.

> **Ek bilgi (derste anlatılmadı):** Taban değiştirme kuralına göre $\log_a n = \dfrac{\log_b n}{\log_b a}$'dır. $\dfrac{1}{\log_b a}$ bir sabit olduğu için farklı tabanlı logaritmalar yalnızca sabit bir çarpanla ayrılır. Büyük-O sabitleri attığından taban büyüme derecesini değiştirmez.

9. Büyük-Omega ve Büyük-Teta

9.1 Büyük-Omega: alt sınır

Soru: Büyük-O için üst sınır demiştik. Büyük-Omega için ne diyeceğiz? **Cevap:** Alt sınır (lower bound).

Soru: Bir fonksiyonun hem üst sınırı hem alt sınırı n ise, yani alt ve üst sınır eşitse, bu hangi sınıra girer? Bir öğrenci "ortalama mı?" diye cevap verdi. Hoca bunun yanlış ama üzerine fikir yürütülmüş bir cevap olduğunu söyledi. **Cevap:** Sıkı sınır (tight bound). Alt ve üst sınırın aynı olduğu durumda sıkı sınır ifadesi kullanılır. Alt sınır için Büyük-Omega, sıkı sınır için Büyük-Teta kullanılır.

Büyük-Omega tanımı, Büyük-O'dakinin eşitsizlik yönü çevrilmiş halidir. $c > 0$ ve n_0 varsa ki $n \geq n_0$ için

$$f(n) \geq c\, g(n)$$

sağlanıyorsa $f(n) \in \Omega(g(n))$'dir. Yani $g(n)$, $f(n)$'nin alt sınırıdır.

Hoca bu derste ağırlıklı olarak Büyük-O kullanılacağını, ama Büyük-Omega ve Büyük-Teta'nın hem soru olarak karşılarına çıkacağını hem de fikir yürütürken kullanılacağını söyledi. Bir algoritmanın alt ve üst sınırı eşitse sıkı sınır olarak ifade etmek tercih edilir. Hocaya göre en güzeli budur, çünkü o zaman algoritmanın net olarak bu karmaşıklığa sahip olduğu söylenebilir.

9.2 Büyük-Teta: sıkı sınır

$c_1 > 0$, $c_2 > 0$ ve n_0 varsa ki $n \geq n_0$ için

$$c_1\, g(n) \leq f(n) \leq c_2\, g(n)$$

sağlanıyorsa $f(n) \in \Theta(g(n))$'dir. Slaytta c_1 ve c_2, c'nin üzerine bir ve iki çizgi konarak gösterilmişti.

Soru: c'nin üzerindeki çentikler 1 ve 2'yi mi belirtiyor, yoksa birinci ve ikinci türevi mi gösteriyor? **Cevap:** Türev yok. Çizgiler yalnızca c_1 ve c_2'yi ayırmak için kullanılmış. Hoca anlatırken c_1 ve c_2 yazdığını söyledi.

Bu tanım, Büyük-Omega'daki $f(n) \geq c\,g(n)$ ile Büyük-O'daki $f(n) \leq c\,g(n)$ eşitsizliklerinin tek bir eşitsizlikte buluşmasıdır. İki farklı sabit gerektiği için tek c yerine c_1 ve c_2 kullanılır. İkisi de sıfırdan büyük olmalıdır.

9.3 Üç gösterimin sezgisel anlamı

- $f(n) \in O(g(n))$: $f(n)$ asimptotik olarak $g(n)$'den küçük ya da eşittir.
- $f(n) \in \Omega(g(n))$: $f(n)$ asimptotik olarak $g(n)$'den büyük ya da eşittir.
- $f(n) \in \Theta(g(n))$: $f(n)$ asimptotik olarak $g(n)$'ye eşittir.

9.4 Slayt örnekleri: $5n^2$

- $5n^2 \in \Omega(n^2)$ **mi?** $f(n) \geq c\,g(n)$ eşitsizliği çözülür: $5n^2 \geq c\,n^2$. $c = 5$, $n_0 = 1$ ile sağlanır. İfade doğrudur.
- $5n^2 \in \Omega(n)$ **mi?** Öğrenciler "diyebiliriz" dedi. Slaytta $c = 1$, $n_0 = 1$ seçilmişti: $5n^2 \geq n$ her $n \geq 1$ için doğrudur.
- $5n^2 \in \Theta(n^2)$ **mi?** Hoca cevabı bir öğrenciye kendi ifadesiyle söyletti. **Cevap:** Hem Büyük-O'ya hem Büyük-Omega'ya uyuyor. Hem alt sınır için hem üst sınır için bir c belirlenebildiğinden Büyük-Teta denebilir. Hoca bunu onayladı: İki eşitsizlik birden sağlanıyorsa $5n^2 \in \Theta(n^2)$'dir. Slaytta $c = 5$, $n_0 = 1$ verilmişti.

10. Logaritma ve Üs Kuralları

Slaytların sonunda matematikten bilinen kurallar hatırlatıldı. Hoca bunların yeri geldikçe kullanılacağını söyledi. Altyazıdan çıkan kurallar şunlardır:

- $\log_b(xy) = \log_b x + \log_b y$
- $\log_b(x/y) = \log_b x - \log_b y$
- $\log_b x^a = a \log_b x$
- $\log_b a = \dfrac{\log_x a}{\log_x b}$
- $a^{b+c} = a^b a^c$
- $a^{bc} = (a^b)^c$
- $a^b / a^c = a^{b-c}$
- $b = a^{\log_a b}$
- $b^c = a^{c \log_a b}$

Hoca bu konuda daha ayrıntılı notları da paylaşacağını söyledi.

11. Sabit Çarpanlar Pratikte Önemlidir

Son slaytta A algoritmasının maliyeti $30000n$, B algoritmasınınki $3n^2$ idi. Hoca slayttaki önemli noktayı öğrencilere söyletmeye çalıştı. Bir öğrenci eleman sayısına göre bir karşılaştırma istendiğini, n 100'e kadar $3n^2$ algoritmasının uygun, 100'den büyükse daha kötü olduğunu söyledi. Başka bir öğrenci iki fonksiyonu eşitleyip sınırı bulduğunu ekledi.

Hoca önce sınırın 100 değil 10000 olduğunu düzeltti:

$$30000n = 3n^2 \Rightarrow n = \frac{30000}{3} = 10000$$

$n < 10000$ için $3n^2 < 30000n$'dir. Yani bu aralıkta B daha az işlem yapar (Python ile kontrol edildi).

Slaytın asıl söylemek istediği şudur: İki algoritmayı teorik olarak kıyaslarken bazı sabitleri devre dışı bırakır, genel bir büyüme derecesi ortaya koyarız. Ama gerçek hayatta, uğraştığınız girdi sayısına bağlı olarak, teoride daha iyi görünen algoritma pratikte dezavantajlı olabilir. Sabitler atılınca n^2, n'den büyük olduğu için B daha verimsiz görünür. Ama sürekli 10000'in altında girdilerle çalışıyorsanız, gerçek işlem sayıları hesaplandığında B'yi tercih etmek A'dan çok daha etkilidir.

> **Hocanın vurgusu:** Her şey teorik olarak ispatlamakla çözülmüyor. Teori yol gösterir, ama ayrıntılı analizdeki sabit katsayıları ve düşük dereceli terimleri problemin boyutuna göre gözden kaçırmayın.

Hoca slaytların önümüzdeki günlerde paylaşılacağını söyledi ve tahtaya geçti.

12. Tahtada Çözülen Örnekler

Tahtada önce üç gösterim özetlendi:

- Büyük-O gösterimi: asimptotik üst sınır, örneğin $O(n)$.
- Büyük-Teta gösterimi: alt ve üst sınırın birleştiği sıkı sınır, örneğin $\Theta(n)$.
- Büyük-Omega gösterimi: asimptotik alt sınır, örneğin $\Omega(n)$.

12.1 $4n^2 + 2n \in O(n)$ mi?

$f(n) = 4n^2 + 2n$, $g(n) = n$. Hoca önce yazılması gereken eşitsizliği sordu. **Cevap:** $f(n) \leq c\,g(n)$, $c > 0$, $n \geq n_0$.

Bir öğrenci "$c = 6$, $n_0 = 1$ için sağlıyor" dedi. Başka bir öğrenci "n^2 olması gerekmez mi?" diye sordu. Hoca sorunun $4n^2 + 2n$'nin $O(n)$ cinsinden ifade edilip edilemeyeceği olduğunu hatırlattı. Öğrenciler ifade edilemeyeceğini söyledi; hoca bunun nasıl gösterileceğini sordu. Çözüm:

- $4n^2 + 2n \leq cn$
- İki taraf n'e bölünür: $4n + 2 \leq c$
- n yalnız bırakılır: $n \leq \dfrac{c-2}{4}$

$c > 0$'dır. Ama bir tarafta değişken n, öbür tarafta sabitler vardır. c'ye hangi sabit konursa konsun, n değişken olduğu için eşitsizlik belli bir n'den sonraki bütün n'ler için sağlanamaz. Bu yüzden "$4n^2 + 2n \in O(n)$ doğru mudur?" sorusunun cevabı **yanlıştır**.

$c = 6$, $n_0 = 1$ önerisi de bunu gösterir: $n = 1$'de $6 \leq 6$ sağlanır, ama $n = 2$'de $20 \leq 12$ sağlanmaz. Python kontrolüne göre $4n^2 + 2n \leq 6n$ yalnız $n = 1$'de doğrudur.

> **Hocanın vurgusu:** "Bunlar çok basit, görüyoruz zaten" diyebilirsiniz. Ama sınavda $4n^2 + 2n$'nin $O(n)$'in elemanı olup olmadığı sorulup çözümleme istendiğinde, evet ya da hayır demek yetmez; bu çözümlemeyi yapmanız beklenir. "Zaten görüyoruz" derseniz ya da yanlışlıkla "sağlanıyor" derseniz cevap yanlış olur.

12.2 $2n + 10 \in O(n)$ ve $n \in O(2n + 10)$

$2n + 10 \in O(n)$**:**

- $2n + 10 \leq cn$, $c > 0$, $n_0 > 0$.
- $10 \leq (c - 2)n$.
- Bir öğrencinin cevabı: $c = 3$, $n_0 = 10$ için sağlanır. $c = 3$ olunca katsayı 1 olur, $n \geq 10$ gerekir. $n_0 = 10$'dan itibaren n büyüdükçe eşitsizlik sağlanır.

Tersi: $n \in O(2n + 10)$ **mi?** Hoca öğrencilerden eşitsizliği yazmalarını istedi. Yazım biraz karışınca tahtada düzeltildi: c parantezin dışında, sağ tarafta olmalıdır.

- $n \leq c\,(2n + 10)$

- c dağıtılır: $n \leq 2cn + 10c$
- $2cn$ sol tarafa atılır: $n - 2cn \leq 10c$, yani $(1 - 2c)\, n \leq 10c$

Bir öğrenci "$c = 1$, $n_0 = 0$ için bile sağlar" dedi. $c = 1$ konunca sol taraf $-n$ olur, sağ taraf 10 olur: $-n \leq 10$. $n = 0$'da sol taraf 0'dır ve eşitsizlik sağlanır. $n = 1$ verilse de sağlanır, bundan sonra da hep sağlanır, çünkü sol taraf negatiftir. Dolayısıyla $n \in O(2n + 10)$ ifadesi doğrudur.

Soru: O zaman sıkı sınır mı olduk? **Cevap:** Bu tek başına sıkı sınır olmadı. Sıkı sınır için ilgili ifadenin hem altını hem üstünü göstermek gerekir. Burada önemli olan, verilen bir fonksiyonun başka bir fonksiyon tarafından Büyük-O cinsinden ifade edilip edilemeyeceğiydi. Sıkı sınır örneği biraz sonra yapılacaktı.

12.3 Büyük-O'nun grafik gösterimi

Hoca tahtaya iki eğri çizdi: $c\, g(n)$ ve $f(n)$. Eğrilerin kesiştiği noktayı işaretleyip bunun neye karşılık geldiğini sordu.

Soru: Bu nokta, az önceki çözümlemedeki hangi parametreye karşılık gelir? Bir öğrenci "sıkı sınır" dedi; hoca hayır dedi. Bir öğrenci $\frac{10c}{1-2c}$ dedi; hoca genel olarak sorduğunu söyledi. **Cevap:** n_0.

$f(n)$ ile $c\, g(n)$ eşitsizliği kurulduğunda, n_0 noktasından sonraki bölgede $f(n)$ her zaman $c\, g(n)$'nin altında kalır. Dolayısıyla $g(n)$, $f(n)$ için bir üst sınırdır. n_0'dan önceki kısımda fonksiyonlar farklı olabilir; bazı noktalarda $f(n)$, bazı noktalarda $c\, g(n)$ üstte kalabilir. Önemli olan belli bir noktadan itibaren koşulun sağlanmasıdır.

12.4 Büyük-Omega'nın grafik gösterimi

Hoca Büyük-Omega tanımını tekrar yazdı: $f(n) \in \Omega(g(n))$, eğer $c > 0$, $n_0 > 0$ ve $n \geq n_0$ için eşitsizlik sağlanıyorsa. **Soru:** Eşitsizlik nasıl yazılmalı? **Cevap:** $f(n) \geq c\, g(n)$.

Sonra iki eğri çizip hangisinin hangi fonksiyona karşılık geldiğini sordu. Cevap: İkinci eğri $c\, g(n)$'dir. Çünkü kesişim noktası n_0'dır ve alt sınırda, bu noktadan sonra $c\, g(n)$ hep $f(n)$'nin altında kalır. Bu da $f(n) \in \Omega(g(n))$ ifadesinin doğru olduğunu gösterir.

12.5 En iyi durum için alt sınır

Soru: Sıralı arama (sequential search) için en iyi durumda (best case) $\Omega(1)$ demek doğru olur mu? **Cevap:** Doğru olur. Sıralı aramada aranan eleman ilk denemede bulunabilir. Hoca bunun için $O(1)$ da denebileceğini ekledi.

Soru: Insertion sort'un en iyi durumu için $\Omega(1)$ denebilir mi? Bir öğrenci önce "n olur, 1 diyemeyiz" dedi, sonra fikrini değiştirdi: Alt sınır belirlediğimiz için en iyi durumdaki n'den daha aşağısı da alt sınırdır; bu yüzden $\Omega(1)$ denebilir. Hoca açıklamaya teşekkür edip asıl soruyu sordu: Hangisini tercih ederiz? **Cevap:** $\Omega(n)$'i, çünkü daha yakındır.

İkisi de en iyi durum için doğrudur. Ama Büyük-O'da olduğu gibi, kendisine en yakın alt sınırı ve en yakın üst sınırı seçmek önemlidir. En yakın alt ve üst sınır aynı fonksiyonsa sıkı sınır söylenebilir.

12.6 Büyük-Teta: tanım ve grafik

İkinci aradan sonra hoca sıkı sınırı tahtaya yazdı:

$$c_1\, g(n) \leq f(n) \leq c_2\, g(n), \quad c_1, c_2 > 0,\ n \geq n_0$$

Hoca burada $c_1 \leq c_2$ olacağını da bir koşul olarak yazdı (bkz. Ders sonrası notlar). Bu sağlanıyorsa $f(n) \in \Theta(g(n))$'dir.

Sonra üç eğri çizip 2, 3, 1 diye numaraladı ve hangisine hangi ifadenin yazılacağını sordu. **Cevap:** En üstteki 2 numaralı eğri $c_2\, g(n)$, en alttaki 1 numaralı eğri $c_1\, g(n)$, aradaki 3 numaralı eğri $f(n)$'dir. Belli bir değerden itibaren $f(n)$, $c_2\, g(n)$'nin altında ve $c_1\, g(n)$'nin üstünde kalır. Bu bölge $f(n) \in \Theta(g(n))$ olduğunu söyler. Koşul sağlandıkça sıkı sınır elde edilir.

12.7 $2n + 5 \in \Theta(n)$

Soru: $2n+5 \in \Theta(n)$ olması için ne sağlanmalı? Bir öğrenci "$c_2 - c_1$ çarpı $g(n)$ mi?" dedi; hoca böyle ifade etmemesini söyledi. **Cevap:** Hem Büyük-O'nun hem Büyük-Omega'nın sağlandığını göstermek gerekir.

Alt sınır (Büyük-Omega):

- $2n + 5 \geq c_1 n$
- $5 \geq (c_1 - 2)\, n$
- Hoca $n_0 = 1$, $c_1 = 3$ önerdi. Bu seçim hatalıdır: $2n + 5 \geq 3n$ yalnız $n \leq 5$ için doğrudur. Doğru seçim $c_1 \leq 2$'dir; örneğin $c_1 = 2$ her $n \geq 1$ için sağlar (bkz. Ders sonrası notlar).

Üst sınır (Büyük-O):

- $2n + 5 \leq c_2 n$
- $5 \leq (c_2 - 2)\, n$
- $n_0 = 1$, $c_2 = 7$ ile sağlanır: $n = 1$'de $5 \leq 5$, sonrasında sol taraf sabit, sağ taraf artar.

İki koşul da sağlandığı için $2n + 5$, $\Theta(n)$ cinsinden ifade edilebilir. Cevap olumludur.

13. Insertion Sort'un Sınırları

13.1 Algoritmanın işleyişi

Hoca `5 7 3 1 2 4 6` dizisini yazıp insertion sort'un sıralama mantığını sordu. Bir öğrenci ilk elemandan başlayıp, solundakinden küçükse yer değiştirerek bu koşul sağlanmayana kadar kaydırdığımızı söyledi. Hoca "ilk elemandan mı başlıyor?" diye sordu. Başka bir öğrenci ikinci elemandan başlandığını söyledi ve adımları anlattı:

Adım	Yerleştirilen eleman	Dizi
Başlangıç		5 7 3 1 2 4 6
1	7 (5'ten küçük değil, yerinde kalır)	5 7 3 1 2 4 6
2	3 (7 ve 5 sağa kayar)	3 5 7 1 2 4 6
3	1 (7, 5, 3 sağa kayar)	1 3 5 7 2 4 6
4	2	1 2 3 5 7 4 6
5	4	1 2 3 4 5 7 6
6	6	1 2 3 4 5 6 7

Derste ilk üç yerleştirme ayrıntılı anlatıldı, 2, 4 ve 6 için yalnız "bu şekilde ilerledik" denildi. Son üç satır Python ile kontrol edildi (ara adım eklendi).

Hocanın benzetmesi iskambil oyunudur: Elinizi dizerken bir kağıt alır, onu olması gereken pozisyona yerleştirirsiniz. Insertion sort'ta da sayıyı alıp olması gereken pozisyona yerleştiririz.

13.2 En kötü ve en iyi durum

Soru: Insertion sort'un karmaşıklığı nedir? Bir öğrenci "en kötü durumda n, en iyi durumda n^2" dedi. Hoca önce hangi durumdan konuşulduğunu belirtmek gerektiğini söyledi: en kötü, ortalama ve en iyi durum vardır. (Öğrencinin cevabında durumlar ters söylenmiştir; aşağıda doğrusu var.)

En kötü durum: Dizi tersten sıralıdır: `7 6 5 4 3 2 1`. Küçükten büyüğe sıralamaya çalışırken her turda elemanlar tek tek kontrol edilerek kaydırılır. Ayrıntılı analiz ileride yapılacak. İç içe iki döngüyle yapıldığı görüldüğünden en kötü durum $O(n^2)$'dir.

Sonra hoca diğer gösterimleri sordu. **Soru:** En kötü durumu Büyük-Omega ile ifade edebilir miyiz, ne yazarız? Bir öğrenci $\Omega(n)$ dedi, ama en iyi durumu düşündüğünü fark etti; hoca henüz en kötü durumda olduklarını hatırlattı. **Cevap:** En kötü durumda $\Omega(n^2)$ yazılabilir. $O(n^2)$ ve $\Omega(n^2)$ birlikte sağlandığından, insertion sort'un en kötü durumu sıkı sınırla $\Theta(n^2)$'dir.

En iyi durum: Dizi zaten sıralıdır. $O(n)$ denebilir, $\Omega(n)$ da denebilir. Dolayısıyla en iyi durum $\Theta(n)$'dir. Hoca ortalama durumu şimdilik bıraktı.

Python ile 7 elemanlı tersten sıralı dizide karşılaştırma sayısı $21 = \frac{7 \cdot 6}{2}$, sıralı dizide $6 = n - 1$ bulundu. Bu sayılar $\Theta(n^2)$ ve $\Theta(n)$ sonuçlarıyla uyumludur.

Hoca dersi burada bitirdi. Aynı günün laboratuvar oturumunda birkaç soru daha çözülecekti.

Örnek Sorular

Bu derste ekran görüntüsü yoktur; sorular hocanın derste sözlü sorduğu sorulardır.

- **(derste sözlü, 00:28:15)** Karmaşıklığı $O(1)$, $O(\log n)$, $O(n)$, $O(n \log n)$, $O(n^3)$, $O(2^n)$ ve $O(n!)$ olan birer algoritma ya da problem örneği verin.
  - Derste verilen cevaplar: sıralı dizide en büyük/en küçük eleman; ikili arama; sırasız dizide verilen sayıdan büyük eleman sayısı; heap sort; matris çarpımı; n elemanlı kümenin alt kümeleri; n elemanın permütasyonları.
- **(derste sözlü, 00:33:04)** $n!$'in büyüme derecesi 2^n'den büyük mü, aynı mı, küçük mü?
  - Derste verilen cevap: Büyüktür. $5! = 120 > 2^5 = 32$ ve fark n büyüdükçe açılır.
- **(derste sözlü, 00:36:40)** 2^n, $n!$ derecesindeki algoritmalar hangi algoritmik yaklaşıma karşılık gelir?
  - Derste verilen cevap: Kaba kuvvet (brute force).
- **(derste sözlü, 00:56:35)** Büyük-Omega neyi gösterir? Alt ve üst sınır aynıysa buna ne denir?
  - Derste verilen cevap: Alt sınır; alt ve üst sınır eşitse sıkı sınır, Θ.
- **(derste sözlü, 01:03:43)** $5n^2 \in \Omega(n^2)$, $5n^2 \in \Omega(n)$ ve $5n^2 \in \Theta(n^2)$ doğru mudur?
  - Derste verilen cevap: Üçü de doğrudur. $\Omega(n^2)$ için $c = 5$, $n_0 = 1$; $\Omega(n)$ için $c = 1$, $n_0 = 1$; $\Theta(n^2)$ için iki eşitsizlik birlikte sağlanır.
- **(derste sözlü, 01:07:17)** A algoritması $30000n$, B algoritması $3n^2$ işlem yapıyor. Son slaytta vurgulanan nokta nedir?
  - Derste verilen cevap: $30000n = 3n^2 \Rightarrow n = 10000$. Girdiler hep 10000'den küçükse asimptotik olarak kötü görünen B pratikte daha hızlıdır.
- **(derste sözlü, 01:14:09)** $4n^2 + 2n \in O(n)$ doğru mudur? Çözümleyerek gösterin.
  - Derste verilen cevap: $4n + 2 \leq c \Rightarrow n \leq \frac{c-2}{4}$. Sabit bir c bütün büyük n'ler için bunu sağlayamaz. İfade yanlıştır.
- **(derste sözlü, 01:20:31)** $2n + 10 \in O(n)$ doğru mudur? Tersi, $n \in O(2n + 10)$ doğru mudur?
  - Derste verilen cevap: İkisi de doğrudur. Birincisi $c = 3$, $n_0 = 10$ ile; ikincisi $c = 1$, $n_0 = 0$ ile sağlanır ($-n \leq 10$).
- **(derste sözlü, 01:26:18)** $f(n)$ ile $c\,g(n)$ eğrilerinin kesişim noktası tanımdaki hangi parametredir?
  - Derste verilen cevap: n_0.
- **(derste sözlü, 01:31:17)** Sıralı aramanın en iyi durumu için $\Omega(1)$ demek doğru mu? Insertion sort'un en iyi durumu için $\Omega(1)$ denebilir mi, hangisi tercih edilir?
  - Derste verilen cevap: İkisi de doğrudur. Insertion sort'un en iyi durumu için en yakın alt sınır olan $\Omega(n)$ tercih edilir.
- **(derste sözlü, 01:49:37)** $2n + 5 \in \Theta(n)$ olduğunu gösterin.
  - Derste verilen cevap: Üst sınır $c_2 = 7$, $n_0 = 1$. Alt sınır için hoca $c_1 = 3$, $n_0 = 1$ dedi; bu hatalıdır.

  Çözüm (derste verilmedi): Alt sınır için $c_1 = 2$ alınır: $2n + 5 \geq 2n$ her $n \geq 1$ için doğrudur. $c_1 = 3$ ile eşitsizlik yalnız $n \leq 5$ için sağlanır (Python ile kontrol edildi). Böylece $c_1 = 2$, $c_2 = 7$, $n_0 = 1$ ile $2n + 5 \in \Theta(n)$'dir.
- **(derste sözlü, 01:55:25)** `5 7 3 1 2 4 6` dizisini insertion sort ile sıralayın. Insertion sort'un en kötü ve en iyi durum karmaşıklığı nedir?
  - Derste verilen cevap: Adımlar Bölüm 13.1'deki tablodadır. En kötü durum (tersten sıralı dizi) $\Theta(n^2)$, en iyi durum (sıralı dizi) $\Theta(n)$.

Ders Sonrası Notlar

Ders sonrası not: Hoca $2n + 5 \in \Theta(n)$ örneğinde alt sınır için $c_1 = 3$, $n_0 = 1$ dedi. $2n + 5 \geq 3n$ yalnız $n \leq 5$ için doğrudur, $n \geq 6$'da bozulur. Doğrusu $c_1 \leq 2$ seçmektir; örneğin $c_1 = 2$ her $n \geq 1$ için sağlar. Sonuç, yani $2n + 5 \in \Theta(n)$, yine doğrudur.

Ders sonrası not: Hoca "n^3 ve 2^n'e geldiğinizde polinom zamanda çözme şansı yok" anlamında konuştu. n^3 polinom zamandır. Polinom olmayan dereceler 2^n ve $n!$ gibi üstel ve faktöriyel derecelerdir.

Ders sonrası not: Hoca Büyük-Teta tanımına $c_1 \leq c_2$ koşulunu da yazdı. Bu, tanımın ayrı bir şartı değildir. $c_1 g(n) \leq f(n) \leq c_2 g(n)$ sağlanıyorsa, $g(n) > 0$ olan noktalarda $c_1 \leq c_2$ kendiliğinden çıkar. Tanım yalnızca $c_1, c_2 > 0$ ister.

Ders sonrası not: Hoca 256 elemanlı dizide n^2 ile $n \log n$ arasındaki farkı "yaklaşık 30 kat" dedi. Tam oran $65536/2048 = 32$'dir.

Dersin Toparlaması

Bu derste büyüme derecesinin sabit çarpanlardan ve düşük dereceli terimlerden etkilenmediği, bu yüzden algoritmaların donanımdan bağımsız olarak büyüme derecesiyle karşılaştırıldığı görüldü. Büyük-O üst sınırı, Büyük-Omega alt sınırı, Büyük-Teta ise ikisinin aynı olduğu sıkı sınırı verir. Bir ifadenin doğruluğu, tanımdaki eşitsizlik kurulup uygun c ve n_0 bulunarak ya da bulunamayacağı gösterilerek ispatlanır. Her zaman fonksiyona en yakın sınır seçilir ve hangi durumun (en kötü, en iyi, ortalama) analiz edildiği belirtilir. Önek ortalamaları örneği, ara sonuçları saklamanın karmaşıklığı $O(n^2)$'den $O(n)$'e indirebildiğini; $30000n$ ile $3n^2$ örneği ise sabitlerin pratikte göz ardı edilemeyeceğini gösterdi.

Insertion sort için en kötü durumda $\Theta(n^2)$, en iyi durumda $\Theta(n)$ bulundu; ayrıntılı analiz ileride yapılacak. Önek ortalamalarındaki "sonucu sakla" fikri, ilerleyen haftalarda görülecek dinamik programlamanın habercisidir. Aynı günün laboratuvar oturumunda ek örnekler çözülecekti.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.